



哈爾濱工業大學 (深圳)  
HARBIN INSTITUTE OF TECHNOLOGY

## 课程报告

开课学期: 2025 夏季

课程名称: 计算机设计与实践

项目名称: 基于 miniRV 的 SoC 设计

项目类型: 综合设计型

课程学时: 48 地点: T2506

学生班级: 6 班

学生学号: 2023311630

学生姓名: 陈广鸣

评阅教师: \_\_\_\_\_

报告成绩: \_\_\_\_\_

实验与创新实践教育中心制

2025 年 6 月

注：本设计报告中各个部分如果页数不够，请同学们自行扩页。原则上一定要把报告写详细，能说明设计的成果、特色和过程。报告应该详细叙述整体设计，以及设计中的每个模块。设计报告将是评定每个人成绩的重要组成部分（**设计内容及报告写作**都作为评分依据）。

设计概述（罗列出所有实现的指令，以及单周期/流水线 CPU 频率）

| 助记符        | 使用语法                | 操作及解释                                                                                                           |
|------------|---------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>R 型</b> |                     |                                                                                                                 |
| add        | add rd,rs1,rs2      | $(rd) \leftarrow (rs1) + (rs2)$                                                                                 |
| sub        | sub rd,rs1,rs2      | $(rd) \leftarrow (rs1) - (rs2)$                                                                                 |
| and        | and rd,rs1,rs2      | $(rd) \leftarrow (rs1) \& (rs2)$                                                                                |
| or         | or rd,rs1,rs2       | $(rd) \leftarrow (rs1)   (rs2)$                                                                                 |
| xor        | xor rd,rs1,rs2      | $(rd) \leftarrow (rs1) \wedge (rs2)$                                                                            |
| sll        | sll rd,rs1,rs2      | $(rd) \leftarrow (rs1) \ll (rs2)$                                                                               |
| srl        | srl rd,rs1,rs2      | $(rd) \leftarrow (rs1) \gg (rs2)$ , 逻辑右移                                                                        |
| sra        | sra rd,rs1,rs2      | $(rd) \leftarrow (rs1) \ggg (rs2)$ , 算术右移                                                                       |
| slt        | slt rd,rs1,rs2      | $(rd) \leftarrow ((rs1) < (rs2))$ , 有符号比较                                                                       |
| sltu       | sltu rd,rs1,rs2     | $(rd) \leftarrow ((rs1) < (rs2))$ , 无符号比较                                                                       |
| <b>I 型</b> |                     |                                                                                                                 |
| addi       | addi rd,rs1,imm     | $(rd) \leftarrow (rs1) + \text{sext}(\text{imm})$                                                               |
| andi       | andi rd,rs1,imm     | $(rd) \leftarrow (rs1) \& \text{sext}(\text{imm})$                                                              |
| ori        | ori rd,rs1,imm      | $(rd) \leftarrow (rs1)   \text{sext}(\text{imm})$                                                               |
| xori       | xori rd,rs1,imm     | $(rd) \leftarrow (rs1) \wedge \text{sext}(\text{imm})$                                                          |
| slli       | slli rd,rs1,imm     | $(rd) \leftarrow (rs1) \ll \text{shamt}$                                                                        |
| srli       | srli rd,rs1,imm     | $(rd) \leftarrow (rs1) \gg \text{shamt}$ , 逻辑右移                                                                 |
| srai       | srai rd,rs1,imm     | $(rd) \leftarrow (rs1) \ggg \text{shamt}$ , 算术右移                                                                |
| slti       | slti rd,rs1,imm     | $(rd) \leftarrow ((rs1) < \text{sext}(\text{imm}))$ , 有符号比较                                                     |
| sltiu      | sltiu rd,rs1,imm    | $(rd) \leftarrow ((rs1) < \text{sext}(\text{imm}))$ , 无符号比较                                                     |
| lb         | lb rd,offset(rs1)   | $(rd) \leftarrow \text{sext}(\text{Mem}[(rs1) + \text{sext}(\text{offset})][7:0])$                              |
| lbu        | lbu rd,offset(rs1)  | $(rd) \leftarrow \text{Mem}[(rs1) + \text{sext}(\text{offset})][7:0]$                                           |
| lh         | lh rd,offset(rs1)   | $(rd) \leftarrow \text{sext}(\text{Mem}[(rs1) + \text{sext}(\text{offset})][15:0])$                             |
| lhu        | lhu rd,offset(rs1)  | $(rd) \leftarrow \text{Mem}[(rs1) + \text{sext}(\text{offset})][15:0]$                                          |
| lw         | lw rd,offset(rs1)   | $(rd) \leftarrow \text{sext}(\text{Mem}[(rs1) + \text{sext}(\text{offset})][31:0])$                             |
| jalr       | jalr rd,offset(rs1) | $t \leftarrow (pc) + 4;$<br>$(pc) \leftarrow ((rs1) + \text{sext}(\text{offset})) \& \sim 1; (rd) \leftarrow t$ |
| <b>S 型</b> |                     |                                                                                                                 |
| sb         | sb rs2,offset(rs1)  | $\text{Mem}[(rs1) + \text{sext}(\text{offset})] \leftarrow (rs2)[7:0]$                                          |
| sh         | sh rs2,offset(rs1)  | $\text{Mem}[(rs1) + \text{sext}(\text{offset})] \leftarrow (rs2)[15:0]$                                         |
| sw         | sw rs2,offset(rs1)  | $\text{Mem}[(rs1) + \text{sext}(\text{offset})] \leftarrow (rs2)[31:0]$                                         |
| <b>B 型</b> |                     |                                                                                                                 |
| beq        | beq rs1,rs2,offset  | if $((rs1) = (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$                                          |
| bne        | bne rs1,rs2,offset  | if $((rs1) \neq (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$                                       |
| blt        | blt rs1,rs2,offset  | if $((rs1) < (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$<br>有符号比较                                 |

|       |                     |                                                        |
|-------|---------------------|--------------------------------------------------------|
| bltu  | bltu rs1,rs2,offset | if ((rs1) < (rs2)) (pc) ← (pc) + sext(offset)<br>无符号比较 |
| bge   | bge rs1,rs2,offset  | if ((rs1) ≥ (rs2)) (pc) ← (pc) + sext(offset)<br>有符号比较 |
| bgeu  | bgeu rs1,rs2,offset | if ((rs1) ≥ (rs2)) (pc) ← (pc) + sext(offset)<br>无符号比较 |
| U 型   |                     |                                                        |
| lui   | lui rd,imm          | (rd) ← sext(imm[31:12] << 12)                          |
| auipc | auipc rd,imm        | (rd) ← (PC) + sext(imm[31:12] << 12)                   |
| J 型   |                     |                                                        |
| jal   | jal rd,offset       | (rd) ← (pc) + 4; (pc) ← (pc) + sext(offset)            |

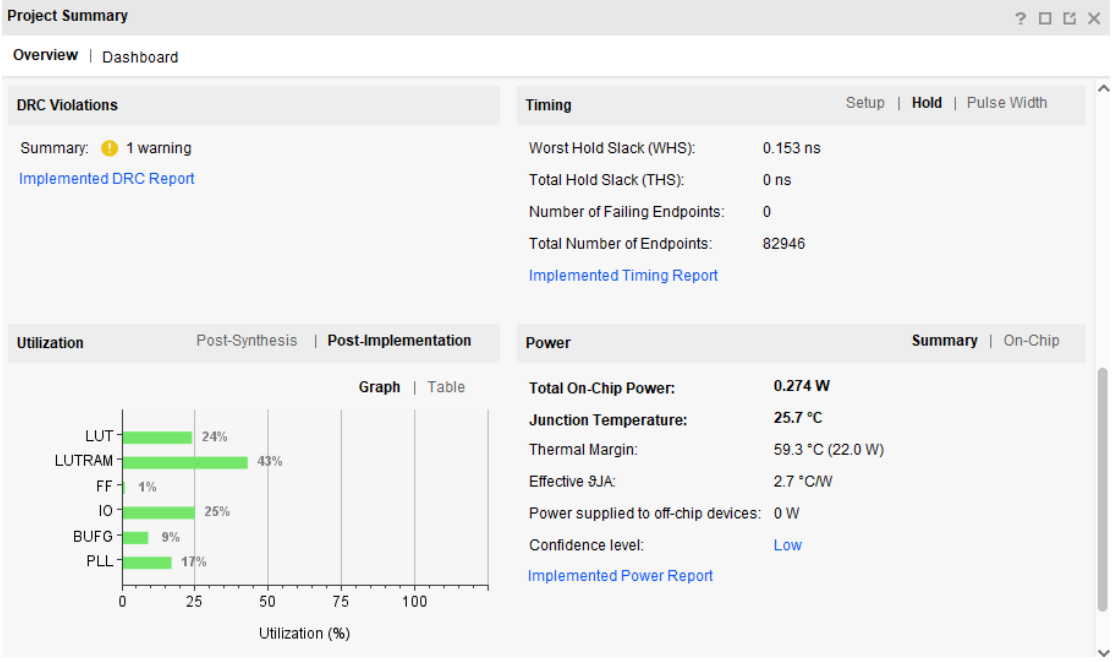
实现指令:单周期与流水线 CPU 均实现 37 条指令  
时钟频率: 单周期 CPU 频率为 40MHz, 流水线 CPU 频率为 88MHz

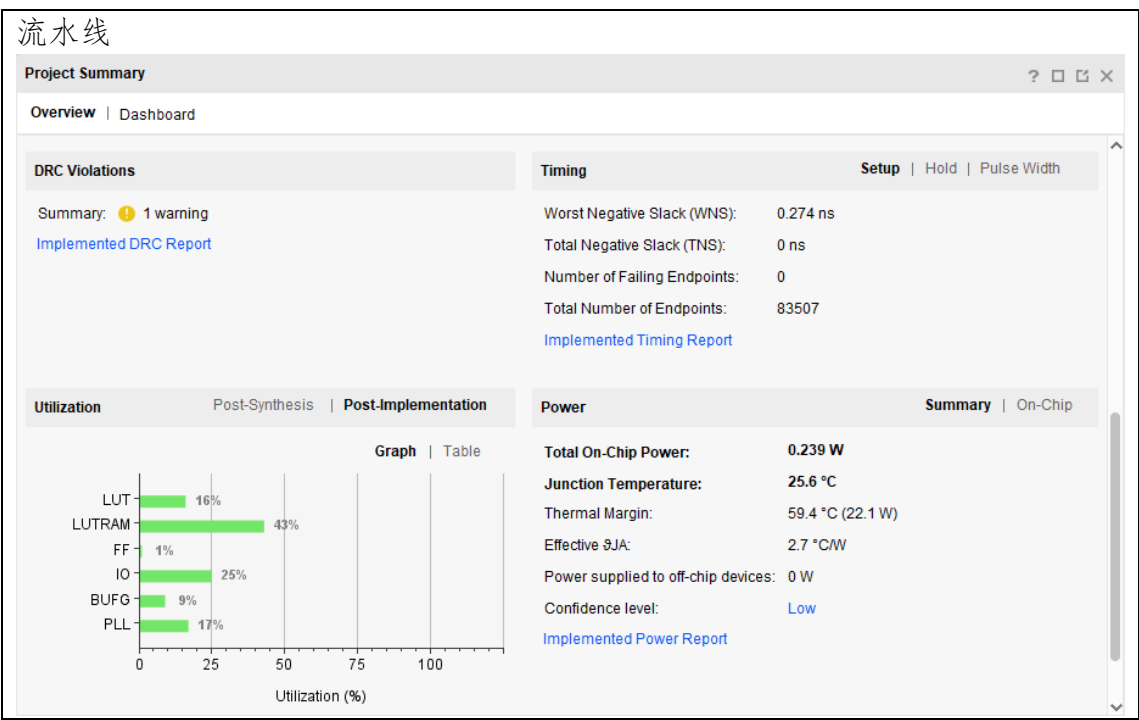
设计的主要特色 (除基本要求以外的设计)

无

资源使用、功耗数据截图 (Post Implementation; 含单周期、流水线 2 个截图)

单周期



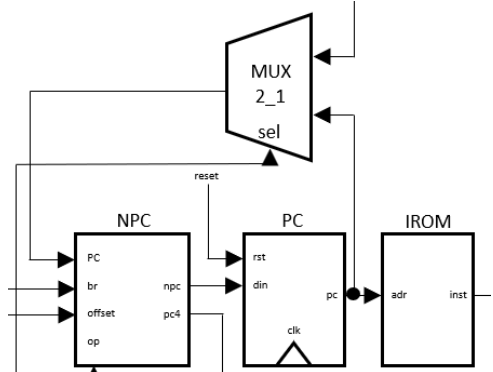




1.2 单周期 CPU 模块详细设计

要求：以表格的形式列出各个部件的接口信号、位宽、功能描述等，并结合图、表、核心代码等形象化工具和手段，详细描述各个部件的关键实现。

1、取指模块（IF）



–NPC：得到下一条指令的地址和 PC+4 的值

|     |            |    |    |                            |
|-----|------------|----|----|----------------------------|
| NPC | op         | 输入 | 2  | NPC 的控制信号，选择 NPC.npc 的数据来源 |
|     | pc         | 输入 | 32 | 当前指令地址                     |
|     | br         | 输入 | 1  | 条件分支是否跳转                   |
|     | alu_result | 输入 | 32 | 执行单元的计算结果                  |
|     | offset     | 输入 | 32 | 跳转的偏移量                     |
|     | pc4        | 输出 | 32 | pc + 4                     |
|     | npc        | 输出 | 32 | 下一条指令地址                    |

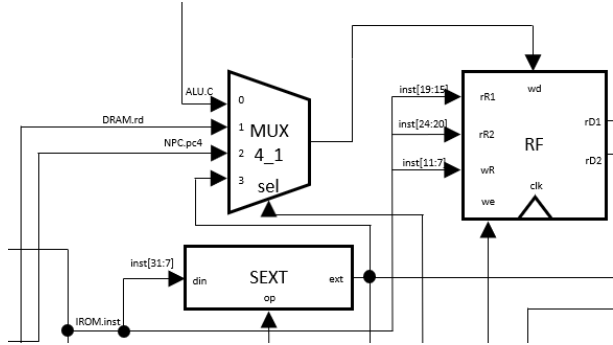
–PC：存储当前指令的地址，指令执行时更新当前 PC 值

|    |     |    |    |         |
|----|-----|----|----|---------|
| PC | rst | 输入 | 1  | 复位信号    |
|    | clk | 输入 | 1  | 时钟信号    |
|    | din | 输入 | 32 | 下一条指令地址 |
|    | pc  | 输出 | 32 | 当前指令地址  |

–IROM：指令存储器，存储指令。根据 PC 输出指令内容

|      |      |    |    |      |
|------|------|----|----|------|
| IROM | adr  | 输入 | 32 | 指令地址 |
|      | inst | 输出 | 32 | 指令   |

2、译码模块（ID）



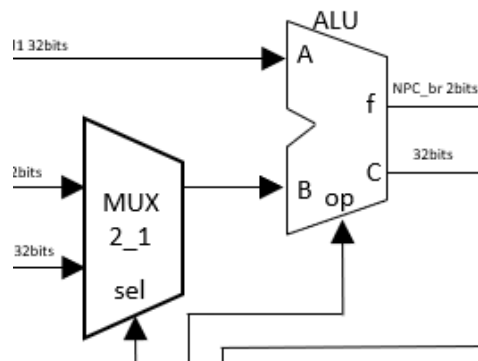
–RF：寄存器堆，包含 32 个 32bit 寄存器

|    |           |    |    |                |
|----|-----------|----|----|----------------|
| RF | clk       | 输入 | 1  | 时钟信号           |
|    | rst       | 输入 | 1  | 复位信号           |
|    | rR1       | 输入 | 5  | 读地址 1          |
|    | rR2       | 输入 | 5  | 读地址 2          |
|    | wR        | 输入 | 5  | 写地址            |
|    | from_alu  | 输入 | 32 | 执行单元的计算后的数据    |
|    | from_dram | 输入 | 32 | 数据寄存器取出的数据     |
|    | from_imm  | 输入 | 32 | 立即数扩展单元扩展后的立即数 |
|    | from_pc4  | 输入 | 32 | pc + 4         |
|    | rf_wsel   | 输入 | 2  | 选择写回寄存器堆的数据来源  |
|    | wd        | 输入 | 1  | 写使能            |
|    | rD1       | 输出 | 32 | 读寄存器数据 1       |
|    | rD2       | 输出 | 32 | 读寄存器数据 2       |
|    | wD        | 输出 | 32 | 写寄存器数据         |

—SEXT: 立即数扩展单元, 根据指令中立即数的格式进行扩展

|      |     |    |    |                    |
|------|-----|----|----|--------------------|
| SEXT | din | 输入 | 25 | 指令中的立即数 inst[31:7] |
|      | op  | 输入 | 3  | 控制扩展方式             |
|      | ext | 输出 | 32 | 扩展后的立即数            |

### 3、执行模块 (EX)



—ALU: 算术逻辑运算单元, 负责完成 CPU 中的算术、逻辑、移位和比较等运算, 生成运算结果、PC 值计算结果和分支指令的跳转结果等

|     |       |    |    |                     |
|-----|-------|----|----|---------------------|
| ALU | data1 | 输入 | 32 | 操作数 1               |
|     | data2 | 输入 | 32 | 操作数 2 (来自 RF.rD2)   |
|     | imm   | 输入 | 32 | 操作数 2 (来自 SEXT.ext) |
|     | sel   | 输入 | 1  | 选择 ALU.B 端口的数据来源    |
|     | op    | 输入 | 4  | 选择运算方式              |
|     | C     | 输入 | 32 | 运算结果                |
|     | bf    | 输入 | 1  | 标志位                 |

### 4、访存模块 (MEM)

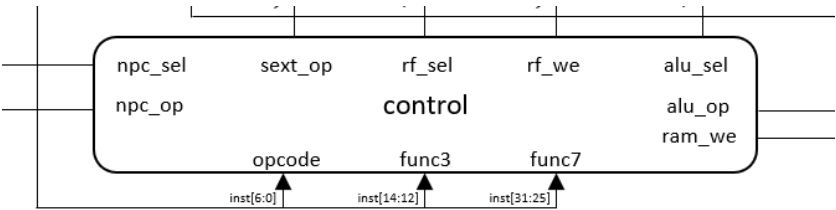
—DRAM: 数据存储器, 根据地址、写使能和时钟, 进行数据的读写操作的时序逻辑部件

|      |      |    |    |      |
|------|------|----|----|------|
| DRAM | clk  | 输入 | 1  | 时钟信号 |
|      | adr  | 输入 | 32 | 访存地址 |
|      | rdo  | 输出 | 32 | 读数据  |
|      | we   | 输入 | 1  | 写使能  |
|      | wdin | 输入 | 32 | 写数据  |

5、写回模块（WB）

→WB\_MUX：选择写回寄存器内容的来源

6、控制模块（CU）



→control：控制器，根据指令中 opcode、func3、func7 的值得到各个器件的控制信号

|         |          |    |    |                    |
|---------|----------|----|----|--------------------|
| control | inst     | 输入 | 32 | 指令                 |
|         | bf       | 输入 | 1  | 条件分支是否跳转           |
|         | npc_op   | 输出 | 2  | 控制 NPC 部件产生指令地址    |
|         | rf_we    | 输出 | 1  | 控制寄存器堆的读写操作        |
|         | rf_wsel  | 输出 | 2  | 控制寄存器堆写回数据的来源      |
|         | sext_op  | 输出 | 3  | 控制 SEXT 部件的符号扩展方式  |
|         | alu_op   | 输出 | 4  | 控制 ALU 的运算类型       |
|         | alub_sel | 输出 | 1  | 控制 ALU 输入口 B 的数据来源 |
|         | ram_we   | 输出 | 1  | 控制 DRAM 的读写操作      |



### 1.3 单周期 CPU 仿真及结果分析

要求：包含逻辑运算、访存、分支跳转三类指令的仿真截图及波形分析；每类指令的截图和分析中，至少包含 1 条具体指令；截图需包含信号名和关键信号。

#### (1) 逻辑运算

—add 指令：

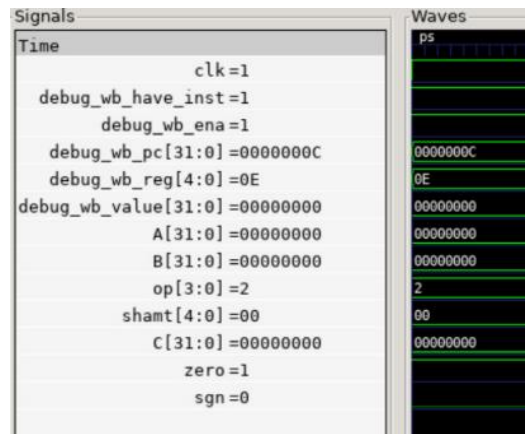
• add, x1, x1, 0

0000004c <reset\_vector>:

```

4: 00000093      addi x1,x0,0
8: 00000113      addi x2,x0,0
c: 00208733      add x14,x1,x2
10: 00000393     addi x7,x0,0
14: 00200193     addi x3,x0,2
18: 4c771663     bne x14,x7,4e4 <fail>

```



根据波形显示,当 pc=00000024 时,从指令存储器里取出指令 00208733, npc 输出下一条顺序执行的指令地址 00000028。将指令分解后将 rR1=x1、rR2=x1、wR=x14 传给寄存器堆,并从相应源寄存器取出数据 rD1=1、rD2=1。运算器执行加法运算(alu\_op=4'b0000)得到结果 C=2,并将结果传回 wD、保存在 x14 寄存器中

—sll 指令：

• sll, from sll.dump

0000007c <test\_7>:

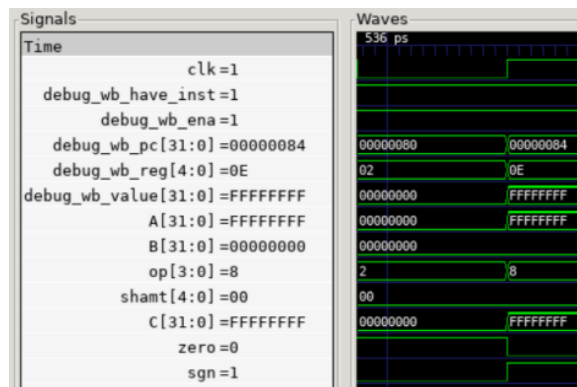
```

7c: fff00093      addi x1,x0,-1
80: 00000113      addi x2,x0,0
84: 00209733      sll x14,x1,x2
88: fff00393      addi x7,x0,-1
8c: 00700193      addi x3,x0,7
90: 4c771263      bne x14,x7,554

```

根据波形显示,当 pc=0000000c 时,从指令存储器里取出指令 0000A703, npc 输出下一条顺序执行的指令地址 00000010。将指令分解后将 rR1=x1、wR=x14 传给寄存器堆,并从相应源寄存器取出数据 rD1=00002000。立即数扩展单元选择 I 型立即数扩展方式(sext\_op=000)并赋值给 ALU.B 端口,运算器执行加法运算(alu\_op=4'b0000)得到结果 C=00002000。结果 C 传至数

据寄存器，从地址 00002000 取出数据 00FF00FF 并保存在 x14 寄存器中。



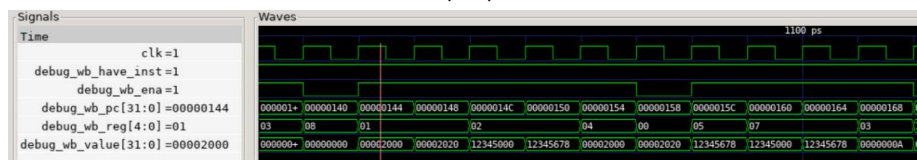
## (2) 访存指令

—lw 指令：

**lw x14, 0(x1)**

00000144 <test\_10>:

|               |                                             |
|---------------|---------------------------------------------|
| 144: 000020b7 | lui x1,0x2                                  |
| 148: 02008093 | addi x1,x1,32 # 2020 <tdat9>                |
| 14c: 12345137 | lui x2,0x12345                              |
| 150: 67810113 | addi x2,x2,1656 # 12345678<_end+0x12343648> |
| 154: fe008213 | addi x4,x1,-32                              |
| 158: 02222023 | sw x2,32(x4) # 20 <reset_vector+0x1c>       |
| 15c: 0000a283 | lw x5,0(x1)                                 |
| 160: 123453b7 | lui x7,0x12345                              |
| 164: 67838393 | addi x7,x7,1656 # 12345678<_end+0x12343648> |
| 168: 00a00193 | addi x3,x0,10                               |
| 16c: 30729e63 | bne x5,x7,488<fail>                         |



x1 = 0000 2020, x2 = 1234 5678, x4 = 0000 2000; 把 x2 的值存到基地址 0000 2000, 偏移 32 的地址, 即 0000 2020; 读取基地址 0000 2020, 偏移 0 的地址, 即 0000 2020 到 x5 中。那么 x5 = 12345678。x7 = 12345678, x5 == x7, 故而不满足 bne, 不跳转, 可以注意到 pc = 164 后是 pc = 168, 故没有发生跳转, 对应正确。

## (3) 分支跳转

—beq 指令：

**beq, from beq.dump**

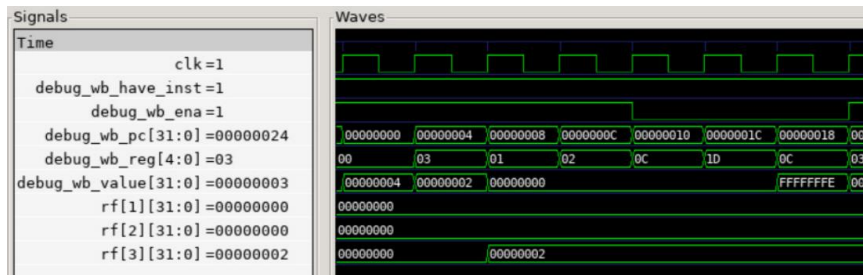
00000004<reset\_vector> :

|              |                                  |
|--------------|----------------------------------|
| 4: 00200193  | addi x3,x0,2                     |
| 8: 00000093  | addi x1,x0,0                     |
| c: 00000113  | addi x2,x0,0                     |
| 10: 00208663 | beq x1,x2,1c <reset_vector+0x18> |
| 14: 2a301863 | bne x0,x3,2c4 <fail>             |

```

18: 00301663      bne x0,x3,24 <test_3>
1c: fe208ee3      beq x1,x2,18 <reset_vector+0x14>
20: 2a301263      bne x0,x3,2c4<fail>

```

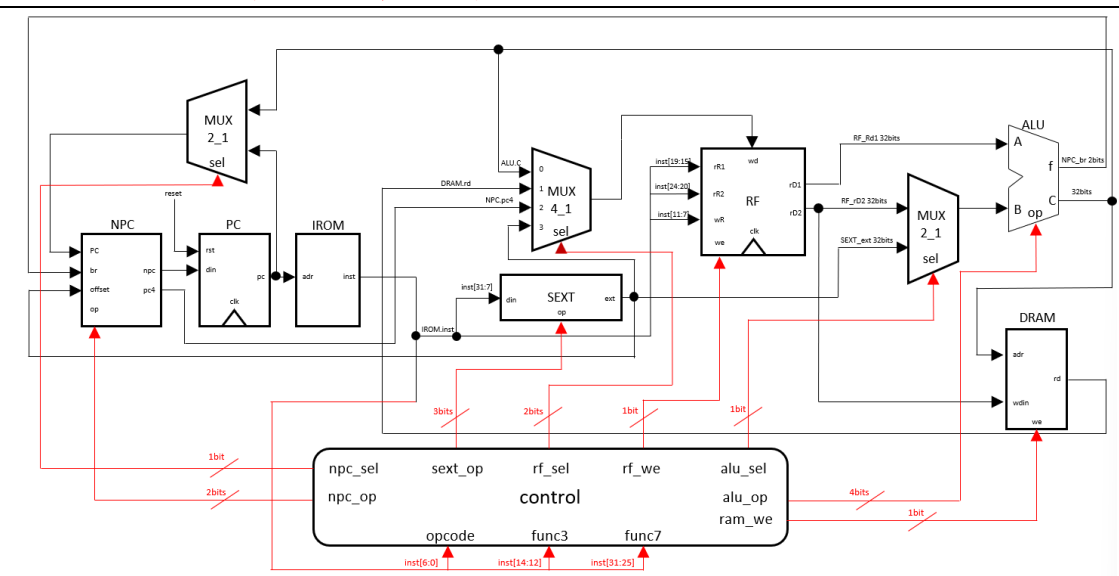


$x3 = 2$ ,  $x1 = 0$ ,  $x2 = 0$ ;  $pc = 10$ :  $x1 == x2$ , 满足 beq, 跳转到  $pc = 1c$ ;  
 $pc = 1c$ :  $x1 == x2$ , 满足 beq, 跳转到  $pc = 18$ ;  $pc = 18$ :  $x0 != x3$ , 满足 bne,  
 跳转到  $pc = 24$ 。

## 2 流水线 CPU 设计与实现

### 2.1 流水线 CPU 数据通路

要求：贴出完整的流水线数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。此外，数据通路图应当能体现出流水线是如何划分的，并用文字阐述每个流水级具备什么功能、需要完成哪些操作。



模块说明：

#### 1、取指模块（IF）

- NPC：得到下一条指令的地址和 PC+4 的值
- PC：存储当前指令的地址，指令执行时更新当前 PC 值
- IROM：指令存储器，存储指令。根据 PC 输出指令内容
- MUX：多路选择器，选择 NPC.pc 的值

#### 2、译码模块（ID）

- RF：寄存器堆，包含 32 个 32bit 寄存器
- SEXT：立即数扩展单元，根据指令中立即数的格式进行扩展
- MUX：多路选择器，选择写回寄存器堆的数据

#### 3、执行模块（EX）

- ALU：算术逻辑运算单元，负责完成 CPU 中的算术、逻辑、移位和比较等运算，生成运算结果、PC 值计算结果和分支指令的跳转结果等
- MUX：多路选择器，选择输入 ALU.B 端口的数据

#### 4、访存模块（MEM）

- DRAM：数据存储器，根据地址、写使能和时钟，进行数据的读写操作的时序逻辑部件

#### 5、写回模块（WB）

- WB\_MUX：选择写回寄存器内容的来源，完成寄存器堆的写操作

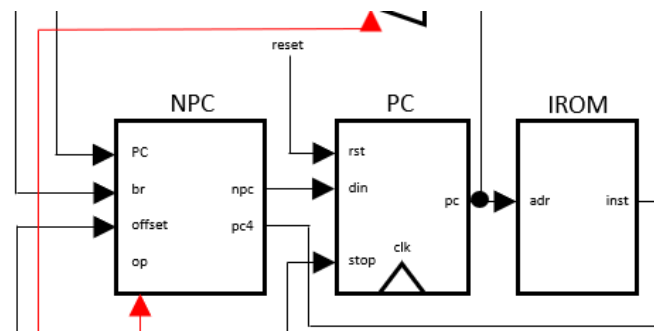
## 6、控制模块 (CU)

—control: 控制器，根据指令中 opcode、func3、func7 的值得到各个器件的控制信号

## 2.2 流水线 CPU 模块详细设计

要求：以表格的形式列出所有与单周期不同的部件的接口信号、位宽、功能描述等，并结合图、表、核心代码等，详细描述这些部件的关键实现。此外，如果实现了冒险控制，必须结合数据通路图，详细说明数据冒险、控制冒险的解决方法。

## 1、取指模块 (IF)



—NPC: 得到下一条指令的地址和 PC+4 的值

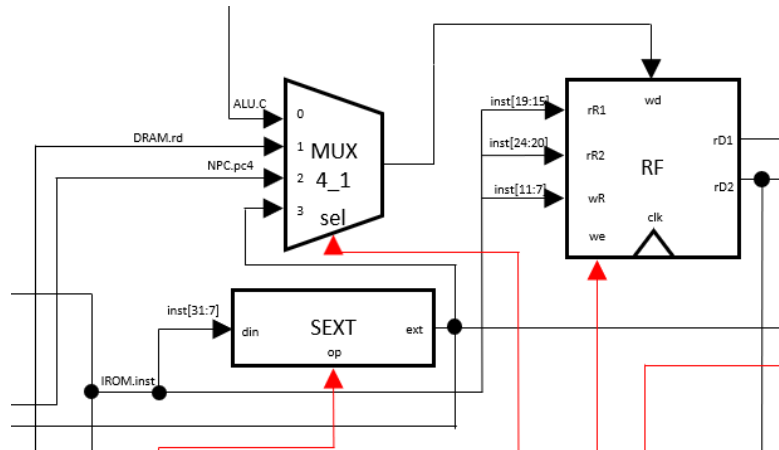
|     |            |    |    |                            |
|-----|------------|----|----|----------------------------|
| NPC | op         | 输入 | 2  | NPC 的控制信号，选择 NPC.npc 的数据来源 |
|     | pc         | 输入 | 32 | 当前指令地址                     |
|     | br         | 输入 | 1  | 条件分支是否跳转                   |
|     | alu_result | 输入 | 32 | 执行单元的计算结果                  |
|     | offset     | 输入 | 32 | 跳转的偏移量                     |
|     | pc4        | 输出 | 32 | pc + 4                     |
|     | npc        | 输出 | 32 | 下一条指令地址                    |

—PC: 存储当前指令的地址，指令执行时更新当前 PC 值

|    |      |    |    |         |
|----|------|----|----|---------|
| PC | rst  | 输入 | 1  | 复位信号    |
|    | clk  | 输入 | 1  | 时钟信号    |
|    | din  | 输入 | 32 | 下一条指令地址 |
|    | pc   | 输出 | 32 | 当前指令地址  |
|    | stop | 输入 | 1  | 停顿信号    |

—IROM: 指令存储器，存储指令。根据 PC 输出指令内容

2、译码模块（ID）



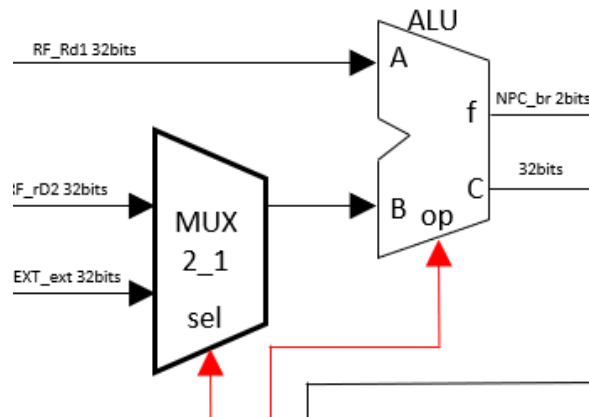
—RF：寄存器堆，包含 32 个 32bit 寄存器

|    |           |    |    |                |
|----|-----------|----|----|----------------|
| RF | clk       | 输入 | 1  | 时钟信号           |
|    | rst       | 输入 | 1  | 复位信号           |
|    | rR1       | 输入 | 5  | 读地址 1          |
|    | rR2       | 输入 | 5  | 读地址 2          |
|    | wR        | 输入 | 5  | 写地址            |
|    | from_alu  | 输入 | 32 | 执行单元的计算后的数据    |
|    | from_dram | 输入 | 32 | 数据寄存器取出的数据     |
|    | from_imm  | 输入 | 32 | 立即数扩展单元扩展后的立即数 |
|    | from_pc4  | 输入 | 32 | pc + 4         |
|    | rf_wsel   | 输入 | 2  | 选择写回寄存器堆的数据来源  |
|    | wd        | 输入 | 1  | 写使能            |
|    | rD1       | 输出 | 32 | 读寄存器数据 1       |
|    | rD2       | 输出 | 32 | 读寄存器数据 2       |
|    | wD        | 输出 | 32 | 写寄存器数据         |

—SEXT：立即数扩展单元，根据指令中立即数的格式进行扩展

|      |     |    |    |                    |
|------|-----|----|----|--------------------|
| SEXT | din | 输入 | 25 | 指令中的立即数 inst[31:7] |
|      | op  | 输入 | 3  | 控制扩展方式             |
|      | ext | 输出 | 32 | 扩展后的立即数            |

3、执行模块（EX）



—ALU: 算术逻辑运算单元, 负责完成 CPU 中的算术、逻辑、移位和比较等运算, 生成运算结果、PC 值计算结果和分支指令的跳转结果等

|     |       |    |    |                     |
|-----|-------|----|----|---------------------|
| ALU | data1 | 输入 | 32 | 操作数 1               |
|     | data2 | 输入 | 32 | 操作数 2 (来自 Rf.rD2)   |
|     | imm   | 输入 | 32 | 操作数 2 (来自 SEXT.ext) |
|     | sel   | 输入 | 1  | 选择 ALU.B 端口的数据来源    |
|     | op    | 输入 | 4  | 选择运算方式              |
|     | C     | 输入 | 32 | 运算结果                |
|     | bf    | 输入 | 1  | 标志位                 |

—MUX: 多路选择器, 选择输入 ALU.B 端口的数据

#### 4、访存模块 (MEM)

—DRAM: 数据存储单元, 根据地址、写使能和时钟, 进行数据的读写操作的时序逻辑部件

|      |      |    |    |      |
|------|------|----|----|------|
| DRAM | clk  | 输入 | 1  | 时钟信号 |
|      | adr  | 输入 | 32 | 访存地址 |
|      | rdo  | 输出 | 32 | 读数据  |
|      | we   | 输入 | 1  | 写使能  |
|      | wdin | 输入 | 32 | 写数据  |

#### 5、控制模块 (CU)

—control: 控制器, 根据指令中 opcode、func3、func7 的值得到各个器件的控制信号

|         |          |    |    |                    |
|---------|----------|----|----|--------------------|
| control | inst     | 输入 | 32 | 指令                 |
|         | bf       | 输入 | 1  | 条件分支是否跳转           |
|         | npc_op   | 输出 | 2  | 控制 NPC 部件产生指令地址    |
|         | rf_we    | 输出 | 1  | 控制寄存器堆的读写操作        |
|         | rf_wsel  | 输出 | 2  | 控制寄存器堆写回数据的来源      |
|         | sext_op  | 输出 | 3  | 控制 SEXT 部件的符号扩展方式  |
|         | alu_op   | 输出 | 4  | 控制 ALU 的运算类型       |
|         | alub_sel | 输出 | 1  | 控制 ALU 输入口 B 的数据来源 |
|         | ram_we   | 输出 | 1  | 控制 DRAM 的读写操作      |

#### 6、冒险检测模块

—ad\_test: 检测并处理数据冒险, 发出停顿、清除的控制信号和相关数据。

检测并处理控制冒险, 发出清除的控制信号。在此将 jal 和 jalr 这两种必跳转指令也视为“控制冒险”, 跳转从 ID 阶段延后至 EX 阶段。

## 2.3 流水线 CPU 仿真及结果分析

要求：包含控制冒险和数据冒险三种情形的仿真截图，以及波形分析。若仅实现了理想流水，则此处贴上理想流水的仿真截图及详细的波形分析。

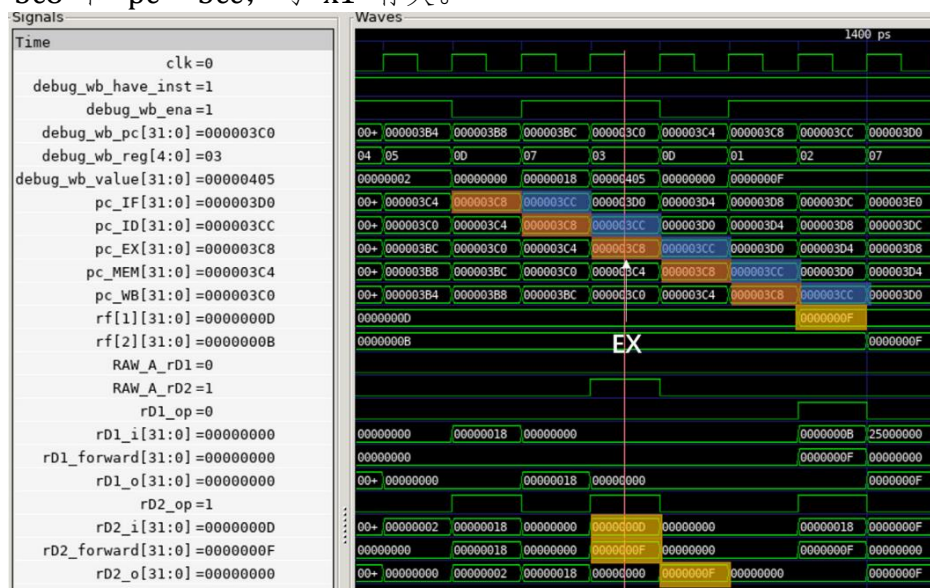
### (1) 数据冒险

—相邻

000003c8 <test\_1035>:

|               |                                          |
|---------------|------------------------------------------|
| 3c8: 00f00093 | addi x1,x0,15                            |
| 3cc: 00100133 | add x2,x0,x1                             |
| 3d0: 00f00393 | addi x7,x0,15                            |
| 3d4: 40b00193 | addi x3,x0,1035                          |
| 3d8: e8711ce3 | bne x2,x7,270 <fail>                     |
| 3dc: 00140413 | addi x8,x8,1                             |
| 3e0: fffff0b7 | lui x1,0xfffff                           |
| 3e4: 0080a023 | sw x8,0(x1) # fffff000 <_end+0xffffaf90> |
| 3e8: 000f80e7 | jalr x1,0(x31)                           |

pc = 3c8 和 pc = 3cc, 与 x1 有关。



x1 在 pc = 3c8 的指令的 WB 阶段结束后，值变为 0000 000f；而 pc = 3cc 的指令的 EX 阶段需要使用到 x1 了，而 x1 的值仍然是原来的 0000 000d，于是就需要使用前递来解决问题。

注意到 pc = 3c8 的指令在 EX 阶段时，pc = 3cc 的指令在 ID 阶段；此时，冒险检测器检测到发生冒险，并对 rD2 进行前递：在 ID/EX 寄存器内，rD2\_i = 0000 000d，这是原本的 x1 的值；前递来的 rD2\_forward = 0000 000f，自然需要选取前递的值作为输出 (rD2\_op = 1)。故在 pc = 3cc 的指令到达 EX 阶段时，从 ID/EX 寄存器得到的 rD2\_o 是 0000000f，成功完成前递，解决数据冒险。

—间隔一条

0000034c : <test\_1020>

|               |              |
|---------------|--------------|
| 34c: 00000213 | addi x4,x0,0 |
|---------------|--------------|

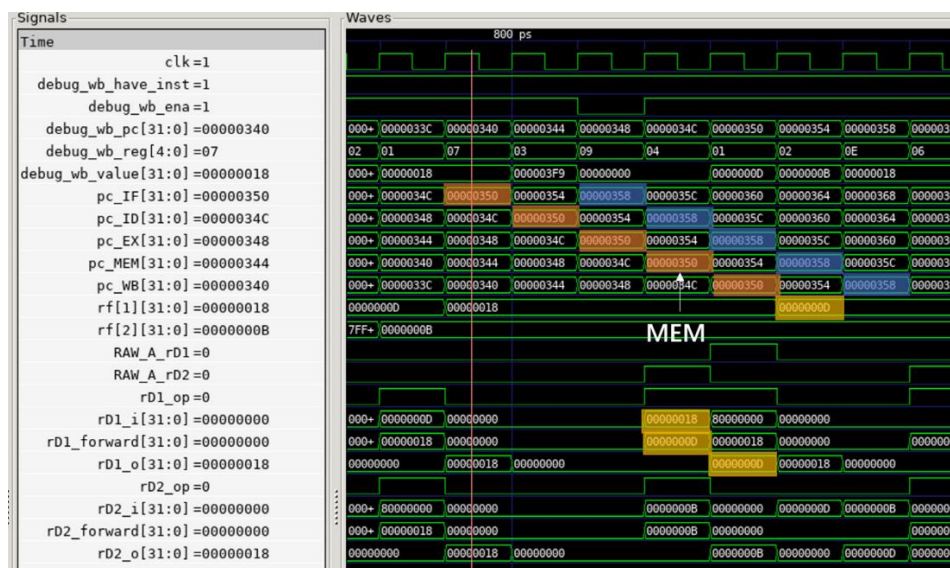


```

350: 00d00093      addi x1,x0,13
354: 00b00113      addi x2,x0,11
358: 00208733      add x14,x1,x2
35c: 00070313      addi x6,x14,0
360: 00120213      addi x4,x4,1 # 1 <_start+0x1>
364: 00200293      addi x5,x0,2
368: fe5214e3      bne x4,x5,350 <test_1020+0x4>
36c: 01800393      addi x7,x0,24
370: 3fc00193      addi x3,x0,1020
374: ee731ee3      bne x6,x7,270<fail>

```

pc = 350 和 pc = 358, 与 x1 有关。



x1 在 pc = 350 的指令的 WB 阶段结束后, 值变为 0000 000d; 而 pc = 358 的指令的 EX 阶段就需要使用到 x1 了, 而 x1 的值仍然是原来的 0000 0018, 于是就需要使用前递来解决问题。

注意到 pc = 350 的指令在 MEM 阶段时, pc = 358 的指令在 ID 阶段; 此时, 冒险检测器检测到发生冒险, 并对 rD1 进行前递: 在 ID/EX 寄存器内, rD1\_i = 0000 0018, 这是原本的 x1 的值; 前递来的 rD1\_forward = 0000 000d, 自然需要选取前递的值作为输出 (rD1\_op = 1)。故在 pc = 358 的指令到达 EX 阶段时, 从 ID/EX 寄存器得到的 rD2\_o 是 0000000d, 成功完成前递, 解决数据冒险。

### 一、间隔两条

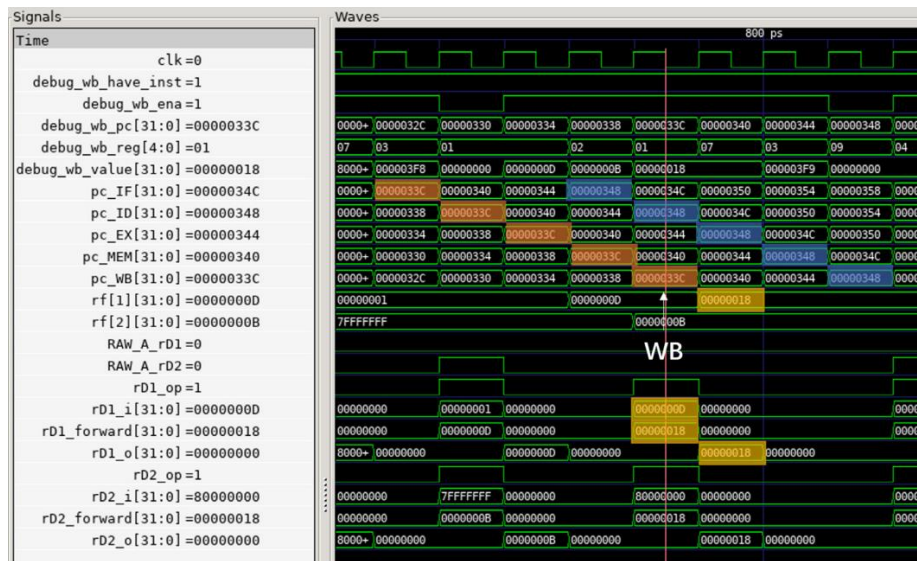
00000334:

```

334: 00d00093      addi x1,x0,13
338: 00b00113      addi x2,x0,11
33c: 002080b3      add x1,x1,x2
340: 01800393      addi x7,x0,24
344: 3f900193      addi x3,x0,1017
348: f27094e3      bne x1,x7,270<fail>

```

pc = 33c 和 pc = 348, 与 x1 有关。



x1 在  $pc = 33c$  的指令的 WB 阶段结束后, 值变为 0000 0018; 而  $pc = 348$  的指令的 EX 阶段就需要使用到 x1 了, 而 x1 的值仍然是原来的 0000 000d, 于是就需要使用前递来解决问题。

注意到  $pc = 33c$  的指令在 WB 阶段时,  $pc = 348$  的指令在 ID 阶段; 此时, 冒险检测器检测到发生冒险, 并对 rD1 进行前递: 在 ID/EX 寄存器内,  $rD1_i = 0000\ 000d$ , 这是原本的 x1 的值; 前递来的  $rD1\_forward = 0000\ 0018$ , 自然需要选取前递的值作为输出 ( $rD1\_op = 1$ )。故在  $pc = 348$  的指令到达 EX 阶段时, 从 ID/EX 寄存器得到的  $rD2_o$  是 00000018, 成功完成前递, 解决数据冒险。

## (2) 控制冒险

00000000<\_start>:

0: 0040006f

jal x0,4 <reset\_vector>

00000004:

4: 02500413

addi x8,x0,37

8: 01841413

slli x8,x8,0x18

c: fffff0b7

lui x1,0xfffff

10: 0080a023

sw x8,0(x1) # fffff000 <\_end+0xffffaf90>

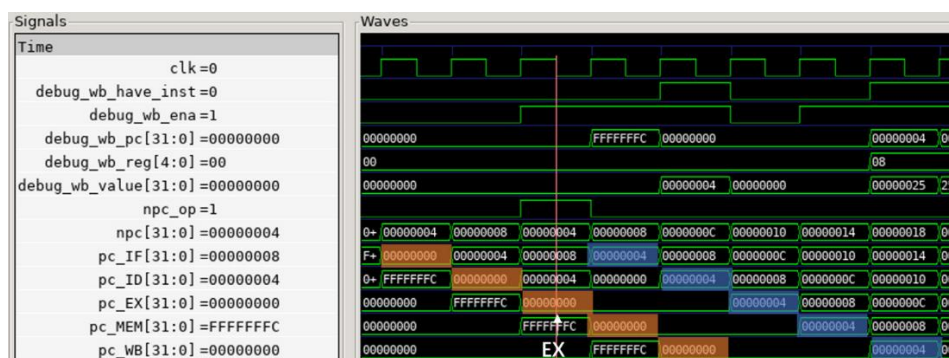
14: 27c00fef

jal x31,290 <n1\_add\_test>

18: 00000013

addi x0,x0,0

对于  $pc = 0$  的 jal 指令:



### 3 设计过程中遇到的问题及解决方法

要求:包括设计过程中遇到的有价值的错误,或测试过程中遇到的有价值的问题。所谓有价值,指的是解决该错误或问题后,能够学到新的知识和技巧,或加深对已有知识的理解和运用。

- (1) 连线冲突 由于在 myCPU 中需要实例化多个模块,且一根信号线可能由多个其他信号决定,所以可能会导致一个输入对应多个输出的情况,造成连线冲突。在这种情况下,需要在 myCPU 模块中进行特殊处理。
- (2) 流水线不同于单周期 CPU,需要在设计初始情况时考虑数据前递以及停顿的问题,因此在设计流水线 CPU 时调整了单周期 CPU 的布局。

## 4 总结

要求：谈谈学完本课程后的个人收获以及对本课程的建议和意见。请在认真总结和思考后填写总结。

虽然计算机组成原理课程奠定了 CPU 的理论基础，但真正动手实践才揭示了其设计的复杂性与细节。计算机设计与实践课程极大地深化了我对 RISC-V CPU，尤其是流水线机制的理解，这种深度是纯理论学习难以企及的宝贵经验。硬件设计，图表先行。实验课让我切身体会到绘制详细数据通路图的重要性。尽管耗时费力，但这些图为后续代码编写和修改提供了不可或缺的蓝图，极大提升了效率。

同时，CPU 设计过程也显著提升了我对 Verilog 语言的掌握。历经无数语法“坑”后，如今编码时已能有效规避常见错误，进行更有条理的硬件描述。然而，下板调试的过程异常艰难。缺乏便捷调试手段，只能根据错误结果反向推测问题所在，这在流水线 CPU 实验中尤甚。耗费大量时间定位一个 bug 的痛苦经历，至今记忆犹新。这也深刻暴露了前期 Trace 测试覆盖不足的问题——若能通过更全面的 Trace 及早发现代码缺陷，后续的调试将高效得多，也不至于如此痛苦。